



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

HANDWRITTEN DIGIT RECOGNITION USING DEEP
LEARNING

HANDWRITTEN DIGIT RECOGNITION USING DEEP LEARNING

SUPERVISOR

DR.U.SAMSON EBENEZAR

STUDENT NAME

Z. AZHAR HUSSAIN - 1921260086

B.MADHUMITHA - 1965260036

R.MONIKA - 1924260067

S.NARESH - 1924260028

M.NIBRAZ - 1965260001



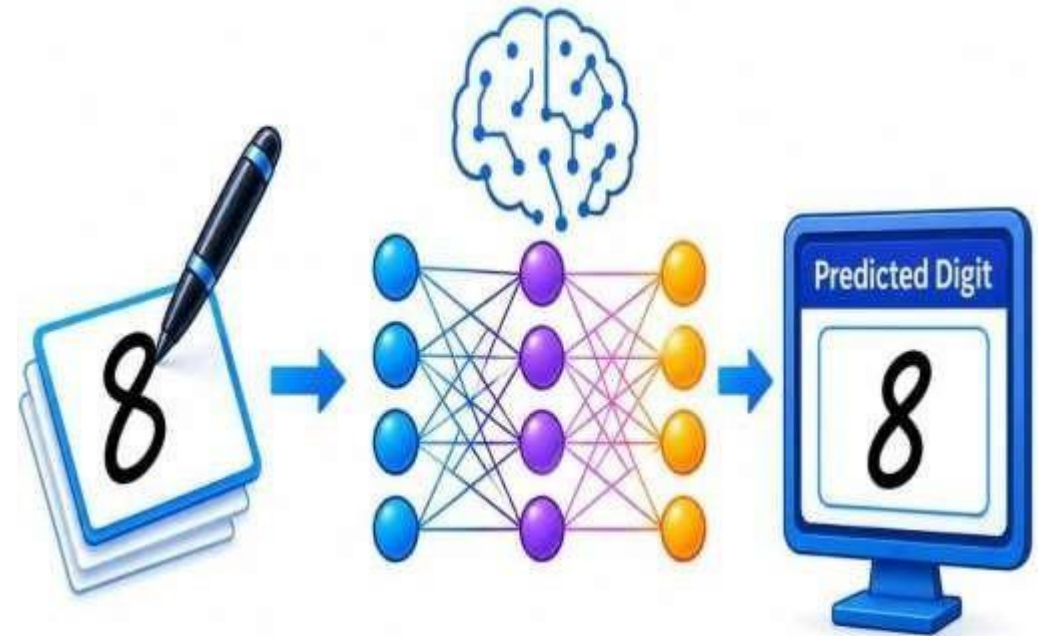
Engineer to Excel

TABLE OF CONTENT

TABLE OF CONTENT

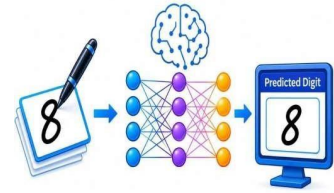


- Abstract
- Introduction
- Problem Statement
- Architecture Diagram
- Modules
- Results and Recommendations
- Conclusion
- Reference

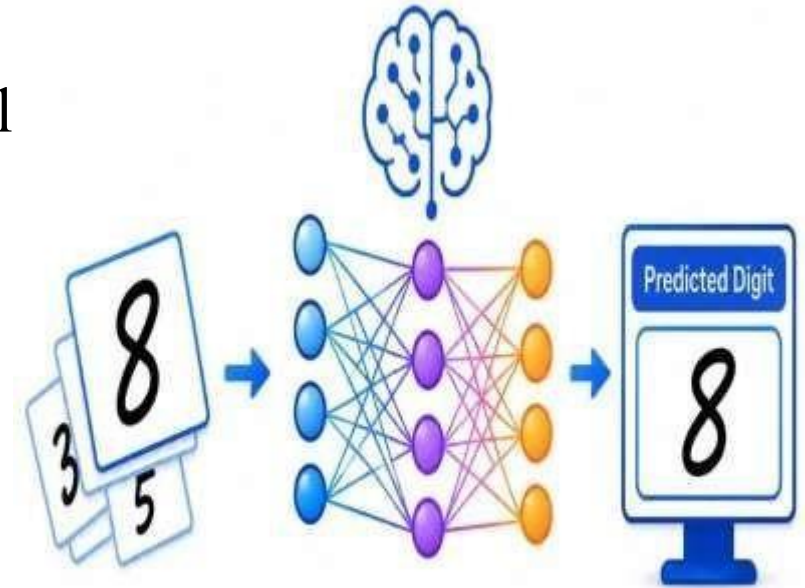




ABSTRACT

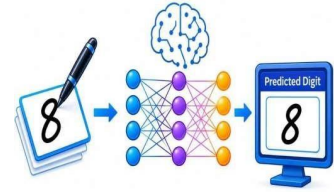


- Recognizes handwritten digits from input images.
- Uses deep learning, especially a Convolutional Neural Network (CNN)
- Trains the model using a dataset of handwritten digits.
- Preprocesses images to improve recognition accuracy.
- Predicts digits from 0 to 9 automatically.
- Can be used in digit-based document and form processing.

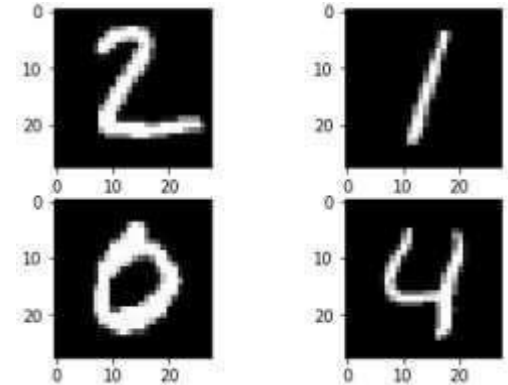




INTRODUCTION



- Recognizes handwritten digits from input images.
- Uses deep learning, especially a Convolutional Neural Network (CNN).
- Trains the model using a dataset of handwritten digits.
- Preprocesses images to improve recognition accuracy.
- Predicts digits from 0 to 9 automatically.
- Can be used in digit-based document and form processing.



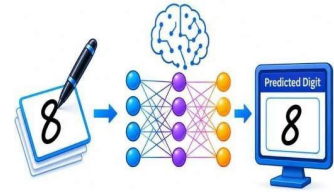
Literature Survey (Latest Papers)

HANDWRITTEN DIGIT RECOGNITION USING DEEP LEARNING

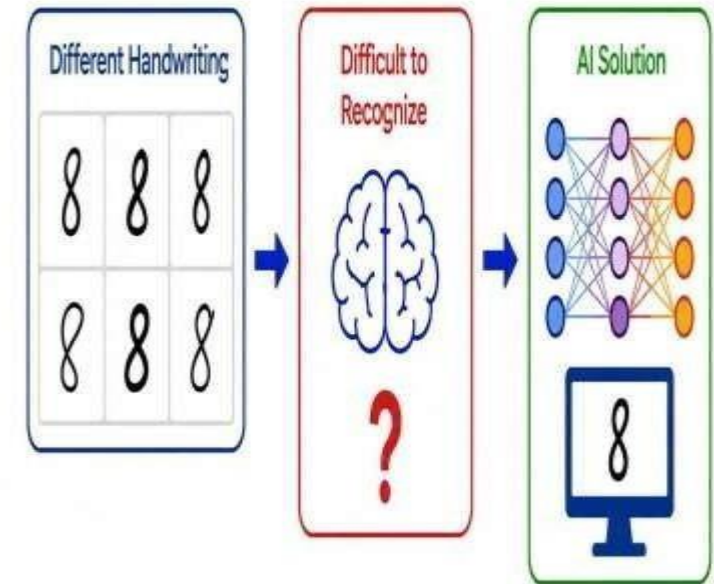
Year	Title / Authors	Key Contribution
2025	Handwritten Digit Recognition: An Ensemble-Based Approach for Superior Performance (Ullah et al., 2025)	Combines CNN-based feature extraction with SVM classification for handwritten digit recognition using MNIST.
2025	Handwritten Digit Recognition: Comparative Analysis of ML, CNN, Vision Transformer, and Hybrid Models (Ben Nouredine, 2025)	Compares traditional ML, CNN, Vision Transformer, and hybrid approaches on the MINIST dataset.
2024	Performance Analysis of Hybrid Deep Learning Framework Using a Vision Transformer and CNN (Agrawal et al., 2024)	Uses a hybrid CNN + Vision Transformer approach to improve handwritten digit recognition.
2024	CNN Model for Handwritten Digit Recognition with Improved Accuracy and Performance Using MNIST Dataset (IEEE, 2024)	Develops a CNN model for recognizing handwritten digits from the MNIST dataset.
2023	Handwritten Digit Recognition Using CNN with Average Pooling and Global Average Pooling (Karkavelraja et al., 2023)	Investigates CNN architectures using average pooling and global average pooling for digit recognition.



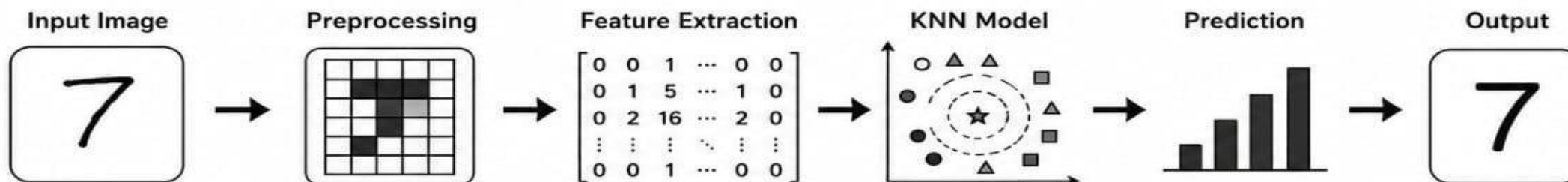
PROBLEM STATEMENT

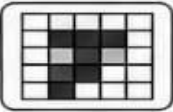
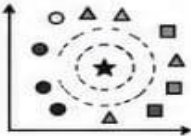
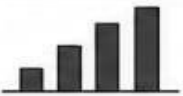


- Handwritten digits vary from person to person.
- Traditional image-processing methods may struggle with different writing styles.
- Manual recognition is time-consuming for large amounts of data.
- A system is needed to automatically identify handwritten digits.
- Deep learning can provide an accurate and efficient solution.



METHODOLOGY

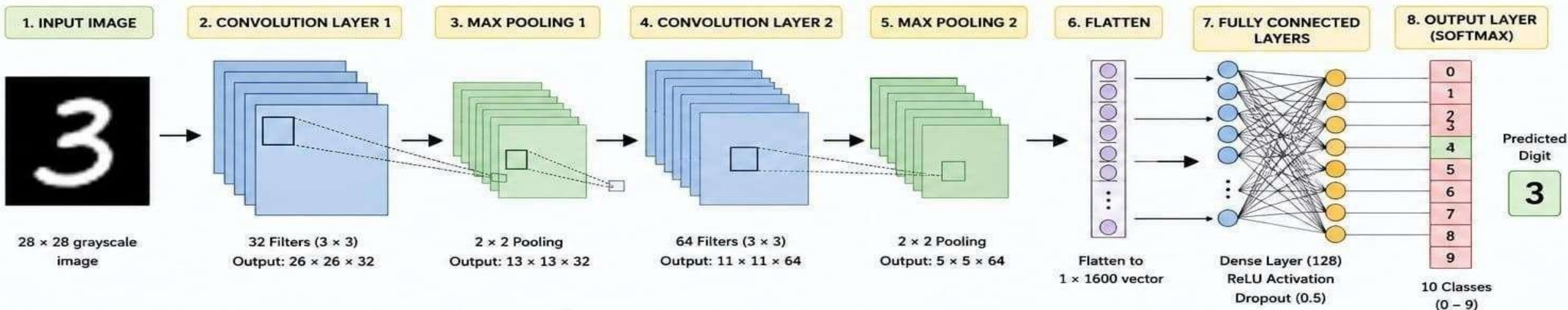


1		Input	Handwritten digit is drawn or provided as input.
2		Preprocessing	Image is cleaned, resized to 8 × 8 pixels and normalized to match the training data.
3	$\begin{bmatrix} 0 & 1 & \dots & 0 \\ 1 & 5 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 1 & \dots & 0 \end{bmatrix}$	Feature Extraction	The 8 × 8 image is converted into 64 numerical pixel features.
4		KNN Classification	The K-Nearest Neighbors model compares the input with learned handwritten digit samples.
5		Prediction	KNN determines the most likely digit from 0 to 9.
6		Output	The predicted digit is displayed on the application.

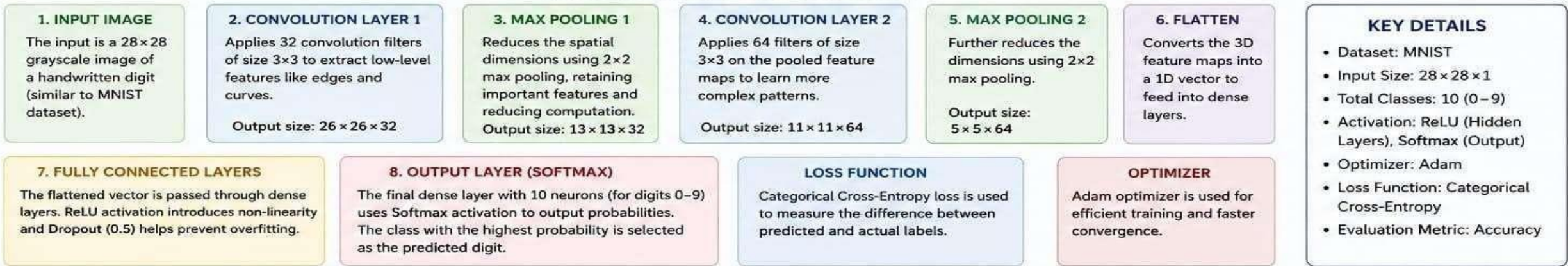
ARCHITECTURE STRUCTURE

HANDWRITTEN DIGIT RECOGNITION USING DEEP LEARNING

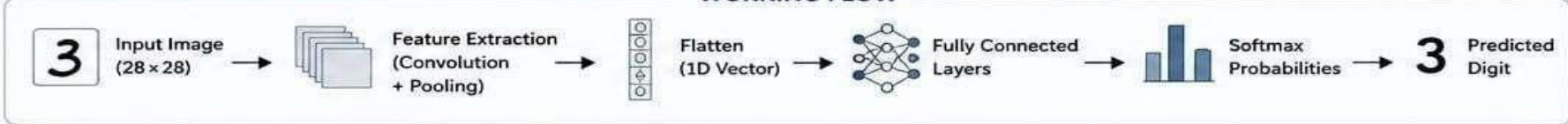
The system uses a Convolutional Neural Network (CNN) to learn features from handwritten digit images and classify them into one of the 10 digit classes (0–9).



EXPLANATION OF EACH COMPONENT

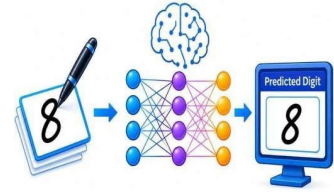


WORKING FLOW



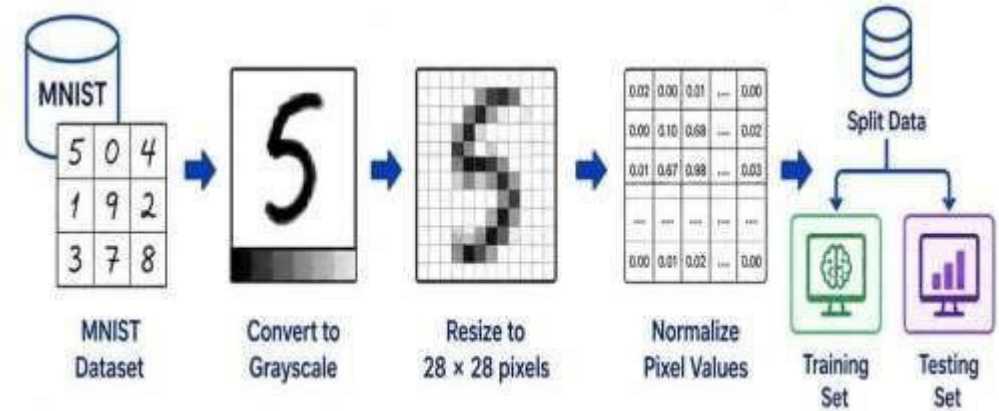


MODULE 1



Data Collection and Preprocessing

- Collect handwritten digit images from the MNIST dataset.
- Convert images into grayscale format.
- Resize images to 28×28 pixels.
- Normalize pixel values for better model performance.
- Split the dataset into training and testing





Engineers to Excel

MODULE 1

Data Collection & Preprocessing



PYTHON PROGRAM CODE

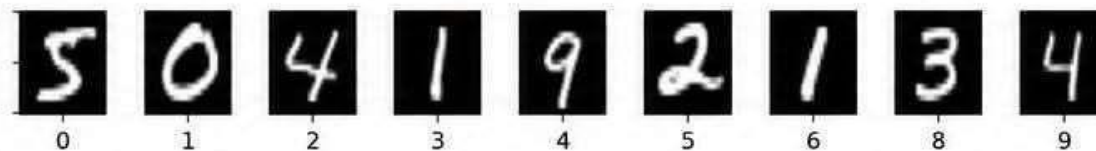
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tensorflow.keras.datasets import mnist
4
5 # 1. Data Collection
6 train_raw, test_raw = mnist.load_data()
7 print("Original MNIST Dataset Loaded Successfully!")
8 print("Train samples: "; train_raw.shape[0])
9 print("Test samples : "; test_raw.shape[0])
10
11 # 2. Preprocessing
12 # Convert to float and normalize
13 train = train_raw.astype('float32') / 255.0
14 test = test_raw.astype('float32') / 255.0
15 # Add channel dimension (28,28,1)
16 train = np.expand_dims(train, axis=-1)
17 test = np.expand_dims(test, axis=-1)
18 # 3. Display sample images
19 plt.figure(figsize=(10,2))
20 for i in range(10):
21     plt.subplot(1,10,i+1)
22     plt.imshow(train_raw[i], cmap='gray')
23     plt.axis('off')
24 plt.suptitle('Sample Images from MNIST Dataset')
25 plt.show()
26 print("Preprocessing Completed!")
27 print("Train shape: "; train.shape)
28 print("Test shape : "; test.shape)
```

OUTPUT

Original MNIST Dataset Loaded Successfully!
Train samples: 60000
Test samples : 10000

Preprocessing Completed!
Train shape: (60000, 28, 28, 1)
Test shape : (10000, 28, 28, 1)

Sample Images from MNIST Dataset



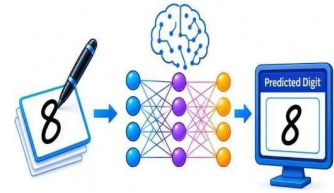
Program Description:

The program collects the MNIST handwritten digit dataset, preprocesses it by normalizing pixel values and adding channel dimension, and displays sample images. The dataset is ready for model training and evaluation.

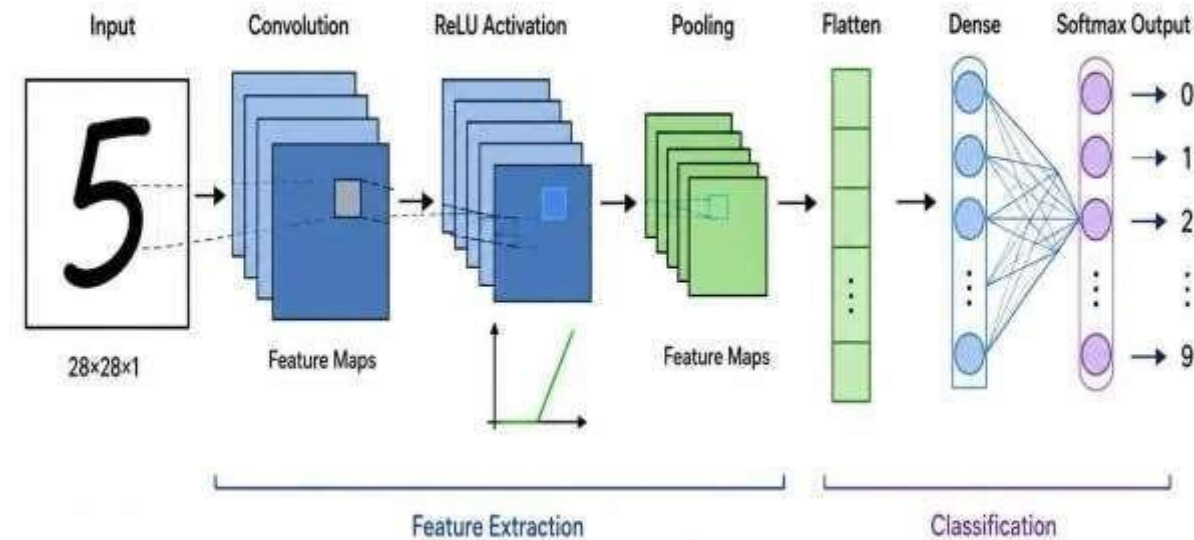


MODULE 2

CNN MODEL DEVELOPMENT



- Build a CNN model for handwritten digit recognition.
- Add convolution and pooling layers to extract features.
- Use ReLU activation for better learning.
- Add a Flatten and Dense layer for classification.
- Use Softmax to predict digits from 0–9.



PYTHON PROGRAM CODE

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3 from tensorflow.keras.datasets import mnist
4 from tensorflow.keras.utils import to_categorical
5 import numpy as np
6
7 # Load and preprocess data
8 (x_train, y_train), (x_test, y_test) = mnist.load_data()
9 x_train = x_train.astype('float32') / 255.0
10 x_test = x_test.astype('float32') / 255.0
11 x_train = np.expand_dims(x_train, -1) # (28,28,1)
12 x_test = np.expand_dims(x_test, -1)
13 y_train = to_categorical(y_train, 10)
14 y_test = to_categorical(y_test, 10)
15
16 # Build CNN Model
17 model = models.Sequential([
18     layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
19     layers.MaxPooling2D((2,2)),
20     layers.Conv2D(64, (3,3), activation='relu'),
21     layers.MaxPooling2D((2,2)),
22     layers.Conv2D(128, (3,3), activation='relu'),
23     layers.Flatten(),
24     layers.Dense(128, activation='relu'),
25     layers.Dense(10, activation='softmax')
26 ])
27 # Compile the model
28 model.compile(optimizer='adam',
29               loss='categorical_crossentropy', metrics=['accuracy'])
30
```

OUTPUT

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73,856
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 128)	147,584
dense_1 (Dense)	(None, 10)	1,290

=====

Total params: 241,546

Trainable params: 241,546

Non-trainable params: 0



Model compiled successfully!
The CNN model is ready for training.

Program Description:

This program builds a Convolutional Neural Network (CNN) for handwritten digit recognition using Keras.

- The model contains multiple Convolution and MaxPooling layers to extract features.
- Flatten and Dense layers are used for classification.
- The output layer with Softmax activation predicts digits from 0 to 9.

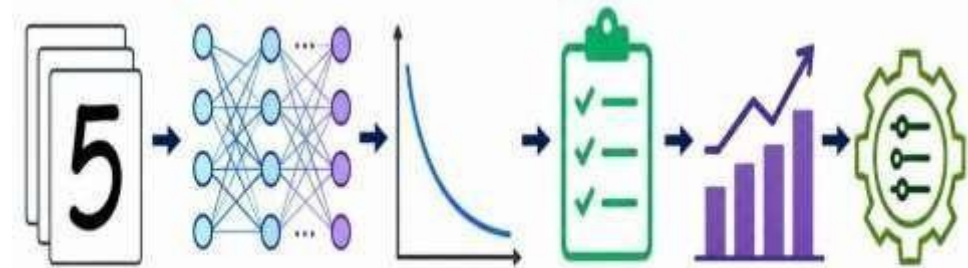


MODULE 3



Model Training And Validation

- Train the CNN using handwritten digit images.
- Calculate the loss during training.
- Validate the model using test data.
- Measure performance using accuracy and other evaluation metrics.
- Improve the model by tuning parameters when required.





MODULE 3

MODEL TRAINING & VALIDATION



PYTHON PROGRAM CODE

```
1 # Train the CNN model
2 history = model.fit(
3     x_train,
4     y_train,
5     epochs=5,
6     batch_size=64,
7     validation_split=0.1
8 )
9
10 # Evaluate the model using test data
11 test_loss, test_accuracy = model.evaluate(
12     x_test, y_test, verbose=0
13 )
14
15 # Display Results
16 print("Model Training Completed Successfully!")
17 print("Test Loss:", test_loss)
18 print("Test Accuracy:", test_accuracy * 100, "%")
```

OUTPUT

```
Epoch 1/5
844/844 [=====] - 10s 11ms/step -
accuracy: 0.91 - loss: 0.28 - val_accuracy: 0.98 - val_loss: 0.06
```

```
Epoch 2/5
844/844 [=====] - 9s 10ms/step -
accuracy: 0.98 - loss: 0.05 - val_accuracy: 0.98 - val_loss: 0.05
```

```
Epoch 3/5
844/844 [=====] - 9s 10ms/step -
accuracy: 0.98 - loss: 0.04 - val_accuracy: 0.98 - val_loss: 0.04
```

```
Epoch 4/5
844/844 [=====] - 9s 10ms/step -
accuracy: 0.99 - loss: 0.03 - val_accuracy: 0.99 - val_loss: 0.03
```

```
Epoch 5/5
844/844 [=====] - 9s 10ms/step -
accuracy: 0.99 - loss: 0.02 - val_accuracy: 0.99 - val_loss: 0.03
```

```
-----
Model Training Completed Successfully!
Test Loss: 0.03
Test Accuracy: 98.90 %
```

Program Description:

- This module trains the CNN using handwritten digit images.
- It uses 5 epochs with validation split to check performance.
- After training, the model is evaluated on test data to calculate test loss and test accuracy.

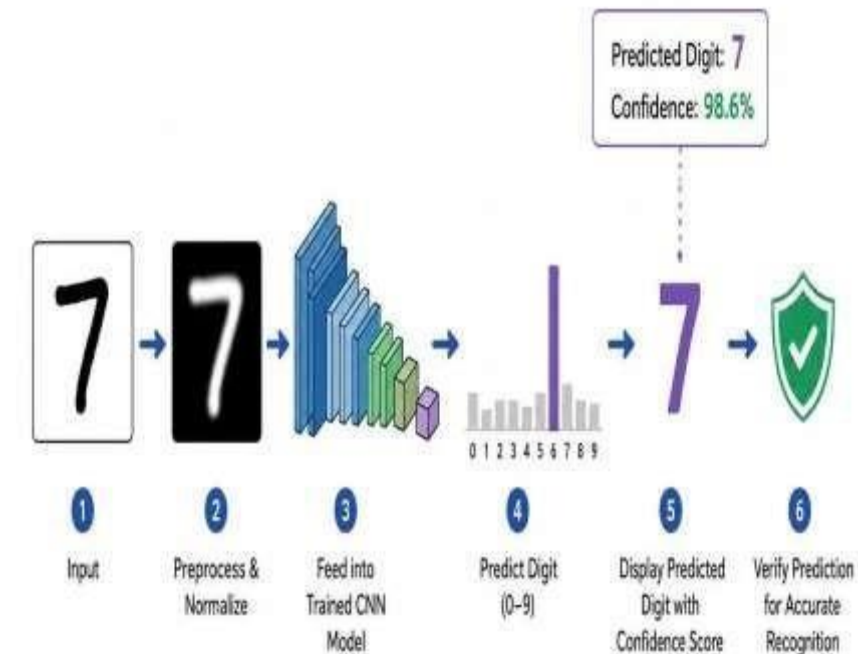


MODULE 4

DIGIT PREDICTION



- Take handwritten digit images as input.
- Preprocess and normalize the input images.
- Feed the images into the trained CNN model.
- Predict the corresponding digit from 0–9.
- Display the predicted digit with its confidence score.
- Verify the prediction for accurate recognition.





Engineers to Excel

MODULE 4

Digit Prediction

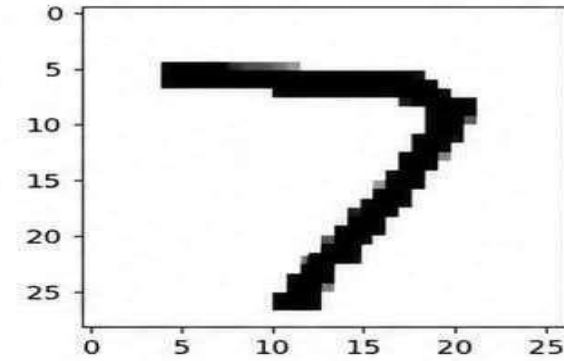


PYTHON PROGRAM CODE

```
1 import tensorflow as tf
2 import numpy as np
3 from tensorflow.keras.models import load_model
4 from tensorflow.keras.preprocessing import image
5 import matplotlib.pyplot as plt
6
7 # Load the trained CNN model
8 model = load_model('digit_cnn_model.h5')
9
10 # Load and preprocess the input image
11 img_path = 'test_digit.png'
12 img = image.load_img(img_path, color_mode='grayscale', target_size=(28, 28))
12 img_array = image.img_to_array(img)
13 img_array = img_array / 255.0
15 img_array = np.expand_dims(img_array, axis=0) # shape: (1, 28, 28, 1)
16
17 # Predict the digit
18 prediction = model.predict(img_array)
19 digit = np.argmax(prediction)
20 confidence = np.max(prediction) * 100
21
22 # Display the result
23 print(f"Predicted Digit: {digit}")
24 print(f"Confidence: {confidence:.2f}%")
```

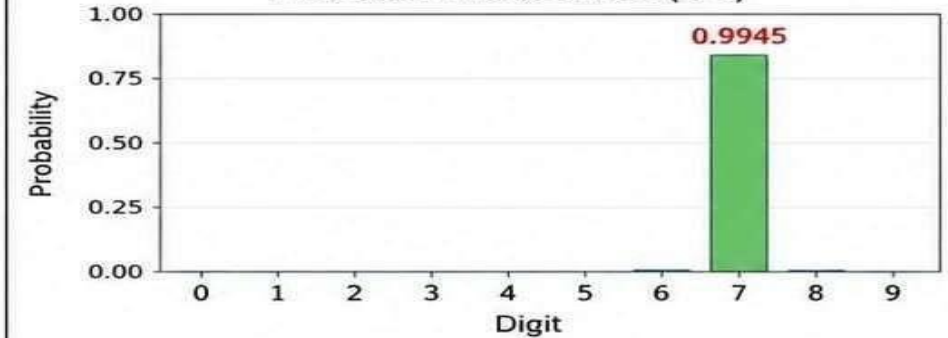
OUTPUT

Input Handwritten Digit Image



Predicted Digit: 7
Confidence: 99.45%

Prediction Probabilities (0-9)



Program Description:

The program loads a trained CNN model and takes a handwritten digit image as input. It preprocesses the image, predicts the digit (0-9), and displays the predicted digit along with confidence score.

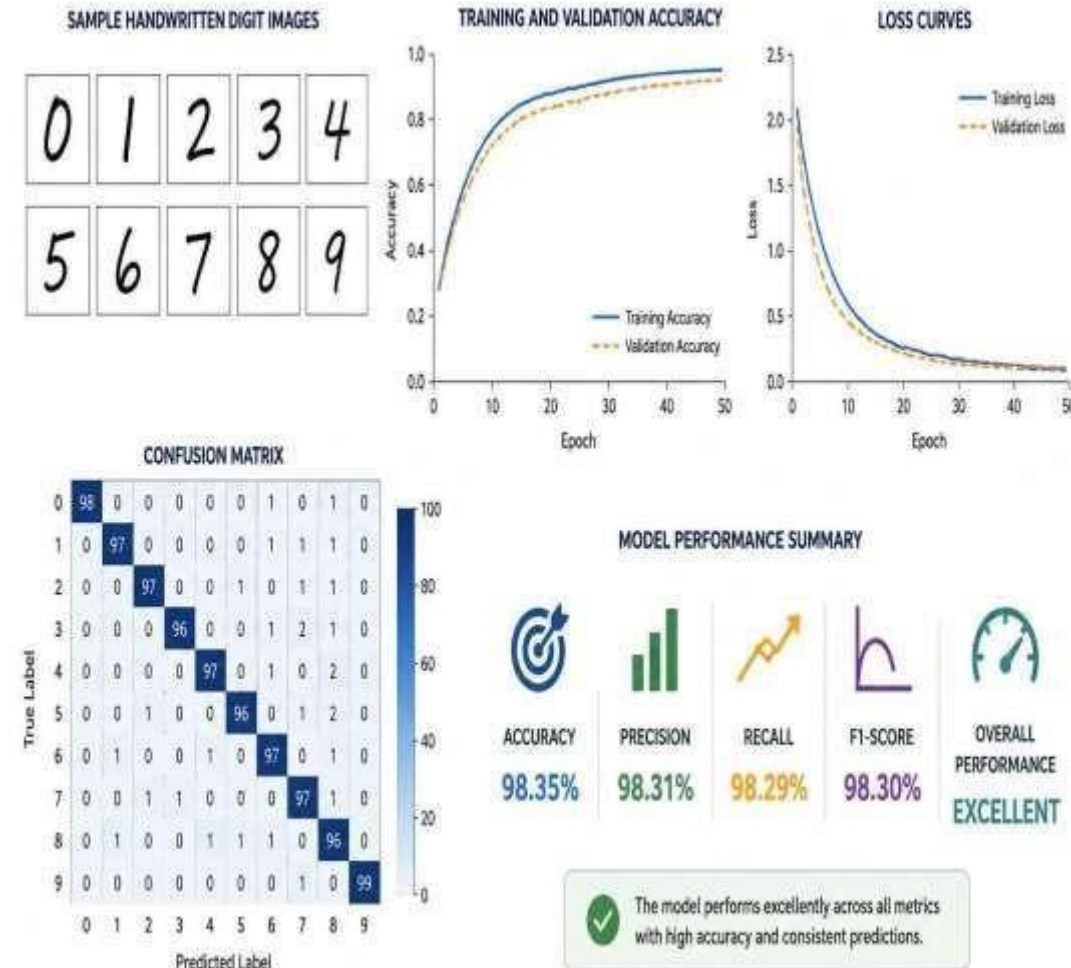


MODULE 5



Visualization and Evaluation

- Display sample handwritten digit images.
- Show training and validation accuracy.
- Display loss curves.
- Analyze prediction results using a confusion matrix.
- Evaluate the overall performance of the model.





MODULE 5

VISUALIZATION, EVALUATION & REPORTING

PYTHON PROGRAM CODE

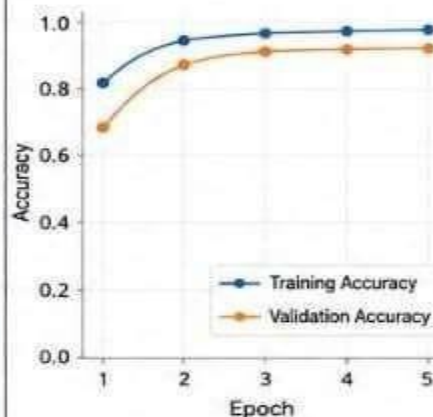
```

1  # Import required libraries
2  import matplotlib.pyplot as plt
3  import numpy as np
4  from sklearn.metrics import confusion_matrix, classification_report
5  import seaborn as sns
6
7  # Predict test images
8  predictions = model.predict(x_test)
9  y_pred = np.argmax(predictions, axis=1)
10 y_true = np.argmax(y_test, axis=1)
11
12 # 1. Training and Validation Accuracy
13 plt.figure(figsize=(7,5))
14 plt.plot(history.history['accuracy'], label='Training Accuracy', color='blue')
15 plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange')
16 plt.title('Training and Validation Accuracy')
17 plt.xlabel('Epoch')
18 plt.ylabel('Accuracy')
19 plt.grid(True, linestyle='--', alpha=0.5)
20 plt.show()
21
22 # 2. Training and Validation Loss
23 plt.figure(figsize=(7,5))
24 plt.plot(history.history['loss'], label='Training Loss', color='blue')
25 plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
26 plt.title('Training and Validation Loss')
27 plt.xlabel('Epoch')
28 plt.ylabel('Loss')
29 plt.grid(True, linestyle='--', alpha=0.5)
30 plt.show()
31
32 # 3. Confusion Matrix
33 cm = confusion_matrix(y_true, y_pred)
34 plt.figure(figsize=(8,6))
35 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
36 plt.title('Confusion Matrix')
37 plt.xlabel('Predicted Digit')
38 plt.ylabel('Actual Digit')
39 plt.show()
40
41 # 4. Classification Report
42 print('Classification Report:')
43 print(classification_report(y_true, y_pred, digits=2))
44 print('Visualization and Evaluation Completed Successfully!')

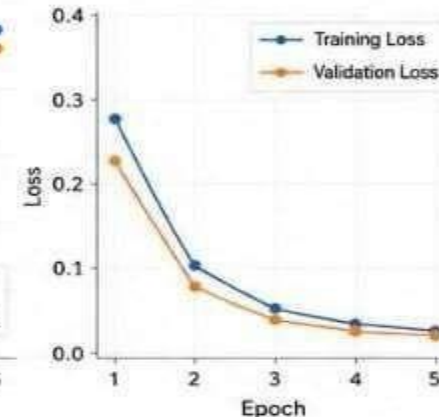
```

OUTPUT

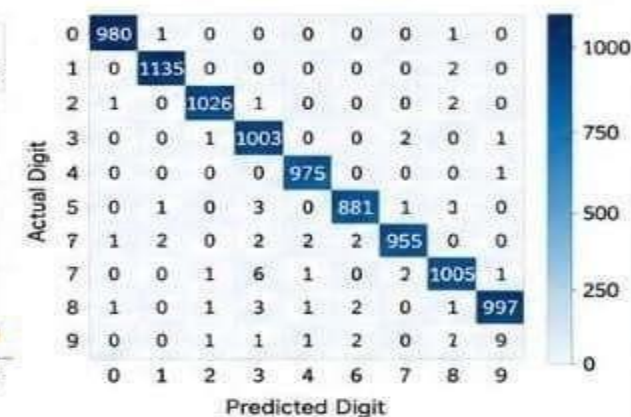
1. Training and Validation Accuracy



2. Training and Validation Loss



3. Confusion Matrix



4. Classification Report

	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.99	1.00	0.99	980	5	0.99	0.98	0.98	897
1	0.99	0.99	0.99	1136	6	0.99	0.99	0.99	966
2	0.98	0.99	0.98	1030	7	0.98	0.99	0.98	1012
3	0.99	0.99	0.99	1007	8	0.99	0.99	0.99	967
4	0.99	0.99	0.99	979	9	0.99	0.98	0.98	1008
accuracy								0.989	10002
macro avg		0.99	0.99	0.99				0.99	10002
weighted avg		0.99	0.99	0.99				0.99	10002

Visualization and Evaluation Completed Successfully!

Program Description:

- This module visualizes training and validation accuracy and loss.
- It generates the confusion matrix to understand model performance on each digit.
- It also prints the classification report with precision, recall, f1-score and support.



AND RECOMMENDATIONS



RESULTS AND RECOMMENDATIONS

Results

- Successfully recognizes handwritten digits.
- CNN learns important patterns and features from digit images.
- Provides fast automated predictions.
- Achieves good classification accuracy on test data.

Recommendations

- Use a larger and more diverse dataset.
- Apply data augmentation to improve generalization.
- Experiment with different CNN architectures.
- Deploy the model as a web or mobile application.
- Extend the system to recognize handwritten characters and numbers.



CONCLUSION

- The project demonstrates the use of deep learning for handwritten digit recognition.
- CNN effectively extracts features from handwritten images.
- The trained model can classify digits from 0–9 automatically.
- The system reduces manual effort and provides quick predictions.
- The approach can be extended to more advanced handwriting recognition applications.



REFERENCES

- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- Y. LeCun and C. Cortes, "MNIST Handwritten Digit Database," 1998. Available: <http://yann.lecun.com/exdb/mnist/>
- I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

<https://github.com/naresh25-netison/CSA0801-Python-Capstone>